Lec 3: Process

Naveen Kumar

Assistant Professor
SVNIT, Surat

February 10, 2026

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

Overview: Processes in Distributed Systems

- **Processes in Distributed Systems**
  - A process is a program in execution
  - Distributed systems introduce challenges beyond OS scheduling
- **Threads**
  - Improve performance in multicore and multiprocessor systems
  - Help structure clients and servers
  - Sometimes replaced by processes for protection and communication
  - Remain essential when performance is critical
- **Virtualization**
  - Enables hardware-independent execution of applications and OS
  - Improves portability and fault isolation
  - Important concept in modern distributed systems

Overview: System Organization and Migration

- **Client–Server Organization**
  - Fundamental model in distributed systems
  - Design issues of clients and servers
  - Object-based systems and Web servers (e.g., Apache)
- **Server Clusters and Wide-Area Systems**
  - Collaborative servers providing the illusion of a single system
  - Examples of wide-area platforms (e.g., PlanetLab)
- **Process and Code Migration**
  - Moving processes or code between machines
  - Supports scalability and dynamic configuration
  - Especially important in wide-area distributed systems

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

Processes vs Threads: Conceptual Foundation

- A **process** is a program in execution with:
  - Its own address space
  - Strong OS-enforced isolation
  - High context-switch overhead
- A **thread** is a lightweight execution unit within a process:
  - Shares address space with other threads
  - Has its own execution flow
  - Faster switching, weaker protection

Process Context and Thread Context

- OS represents execution using contexts
- **Process context** includes:
  - Address space and memory mappings
  - Open files and resources
  - Privileges and accounting data
- **Thread context** includes:
  - Program counter, registers, stack
  - Minimal scheduling information

Cost of Process Isolation

- Strong isolation improves safety but introduces overhead
- Each process has:
  - Separate virtual address space
  - Memory protection via MMU
- Overheads include:
  - Expensive context switches
  - TLB flushes
  - Memory remapping

Why Threads Exist

- Processes are often too large and expensive to use.
- Threads reduce overhead by:
  - Sharing memory
  - Avoiding repeated address space switches
- Benefits:
  - Higher performance
  - Better CPU utilization
  - Easier concurrency

Threads in Single-Machine Systems

- Single-threaded process blocks on I/O
- Multithreading enables:
  - Overlapping I/O and computation
  - Responsive user interfaces
- Example:
  - Spreadsheet with threads for UI, computation, and autosave

Threads and Parallelism

- Threads exploit multiprocessor and multicore systems
- Multiple threads may execute simultaneously
- Shared memory enables fast communication
- Same program scales automatically with more cores

Threads vs Processes in Large Applications

- Large systems often use multiple processes
- Interprocess communication (IPC) requires kernel support:
  - Pipes, message queues, shared memory
- IPC causes:
  - Extra context switches
  - Performance overhead

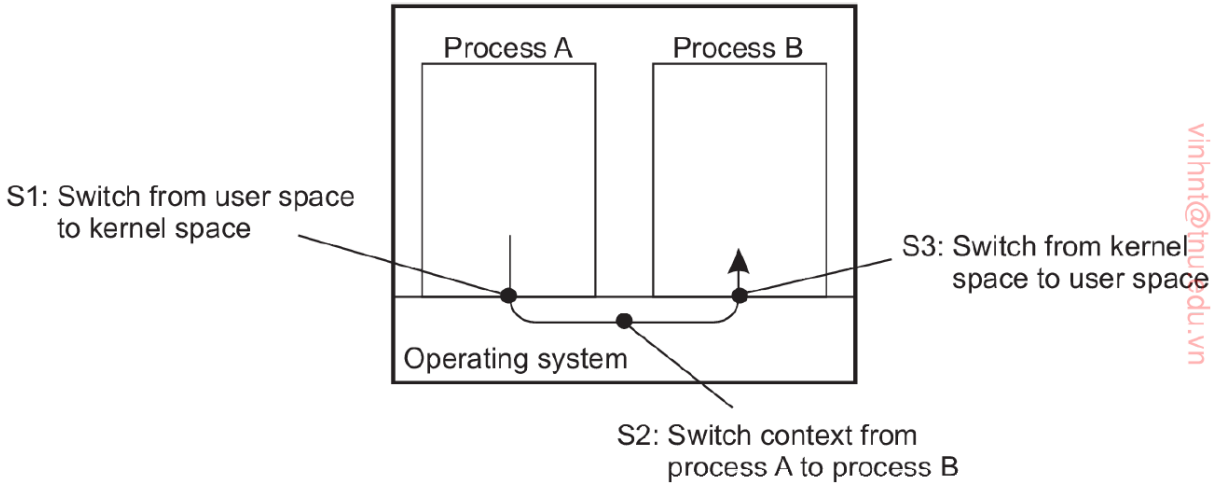


Figure 3.1: Context switching as the result of IPC.

vinhnt@tnu.edu.vn

Why Threads Are Faster Than IPC

- Threads communicate via shared memory
- No kernel involvement required
- Thread switching may occur in user space
- Results in:
  - Lower latency
  - Higher throughput

Context Switching Overhead

- **Direct costs**:
  - Saving and restoring CPU state
  - Interrupt handling
- **Indirect costs**:
  - Cache pollution
  - Reduced memory locality
- Cache effects dominate performance loss

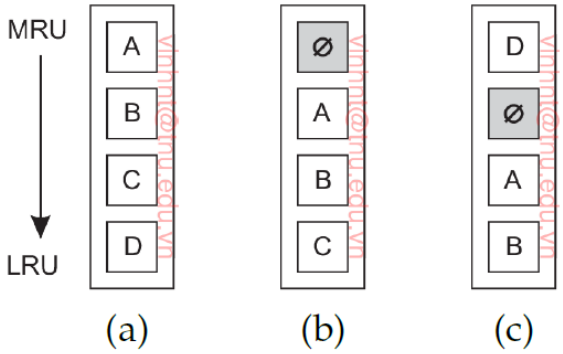


Figure 3.2: The organization of the cache when dealing with interrupts: (a) before the context switch, (b) after the context switch, and (c) after accessing block D. (Adapted from Liu and Solihin [2010].)

User-Level vs Kernel-Level Threads

- Threads can be managed at different levels of the system
- The management level determines performance and parallelism

User-Level Threads

- Managed by a **user-space thread library**
- Kernel is **unaware** of individual threads
- Thread creation and switching are **fast**
- A blocking system call blocks the **entire process**

Kernel-Level Threads

- Managed directly by the **operating system kernel**
- Kernel schedules each thread independently
- Supports true parallelism on multicore processors
- Thread operations incur **higher overhead**

Thread Mapping Models

- Thread models define how **user-level threads**
- are mapped to **kernel-level threads**
- **Many-to-One**
  - Multiple user threads mapped to a single kernel thread
  - Implemented using user-level thread libraries
- **One-to-One**
  - Each user thread mapped to a separate kernel thread
  - Implemented using kernel-level threads
- **Many-to-Many**
  - Multiple user threads multiplexed over multiple kernel threads
  - Combines performance and parallelism

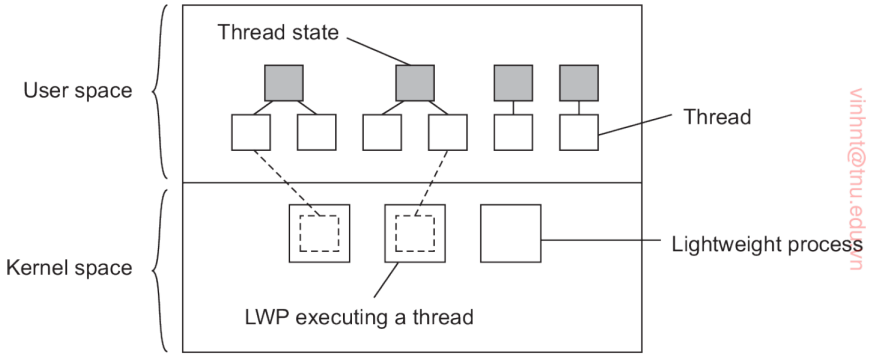


Figure 3.3: Combining kernel-level processes and user-level threads.

Threads in Distributed Systems

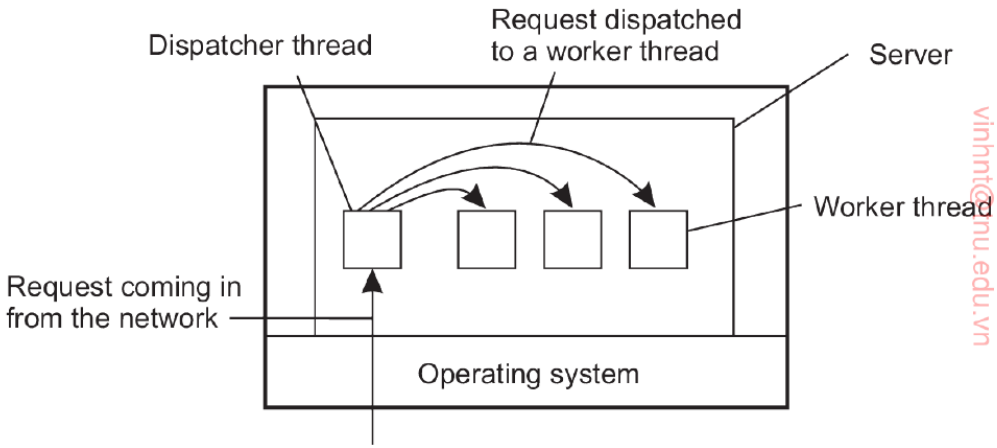
- Distributed systems involve high communication latency
- Blocking calls are common
- Threads allow:
  - Waiting for communication without blocking the process
  - Concurrent handling of multiple connections

Multithreaded Clients

- Goal: hide communication latency from users
- Example: Web browsers
  - Fetch HTML first
  - Load images and scripts in parallel
- Threads improve responsiveness and perceived performance

Multithreaded Servers

- Servers often handle many concurrent clients
- Blocking I/O is common (disk, network)
- Multithreading allows:
  - One thread per request
  - Other requests to proceed during blocking I/O



vinhnt@nu.edu.vn

Figure 3.4: A multithreaded server organized in a dispatcher/worker model.

Why Threads Are Preferred

- Preserve sequential programming model
- Allow blocking system calls
- Enable parallelism on multicore systems
- Balance simplicity and performance

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

From Parallelism to Virtualization

- Threads and processes create the illusion of parallel execution
- On a single CPU, only one instruction executes at a time
- Rapid context switching creates an **illusion of concurrency**
- This idea extends beyond CPUs to other resources
- Leads to **resource virtualization**

Why Virtualization Exists

- Hardware and low-level systems evolve rapidly
- Applications and middleware evolve slowly
- Result: software outlives its hardware platform
- Virtualization allows:
  - Legacy software to run on new hardware
  - Stable interfaces over time

What Is Virtualized?

- Computer systems expose interfaces to software
- Interfaces exist at multiple levels:
  - Instruction Set Architecture (ISA)
  - System calls
  - Libraries and APIs
- Virtualization:
  - Replaces or extends an interface
  - Mimics the behavior of another system

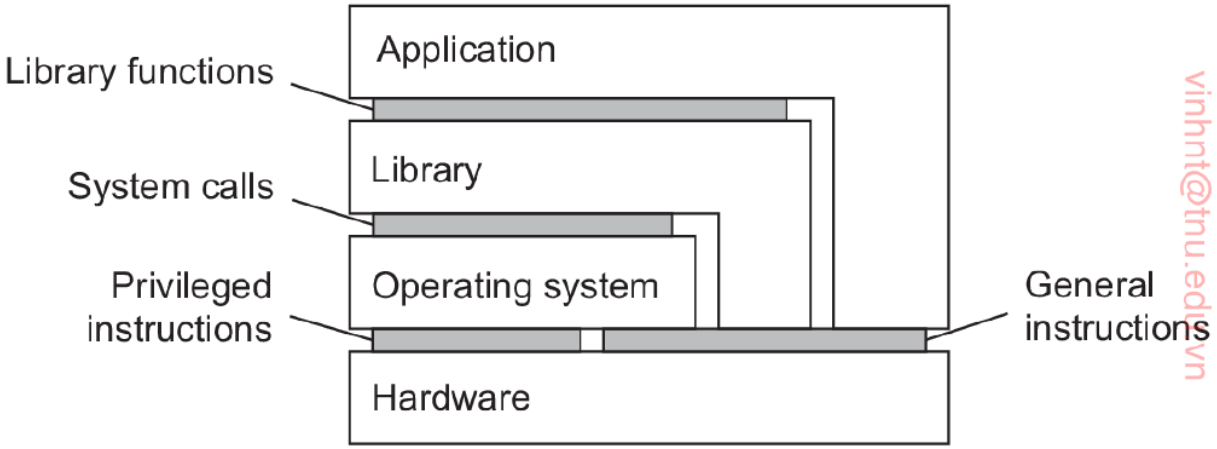


Figure 3.7: Various interfaces offered by computer systems.

vinhnt@tnu.edu.vn

Virtualizing One System on Another

- Virtualization inserts a layer between software and hardware
- This layer emulates a different system
- Applications interact with a **virtual interface**
- Physical hardware details are hidden

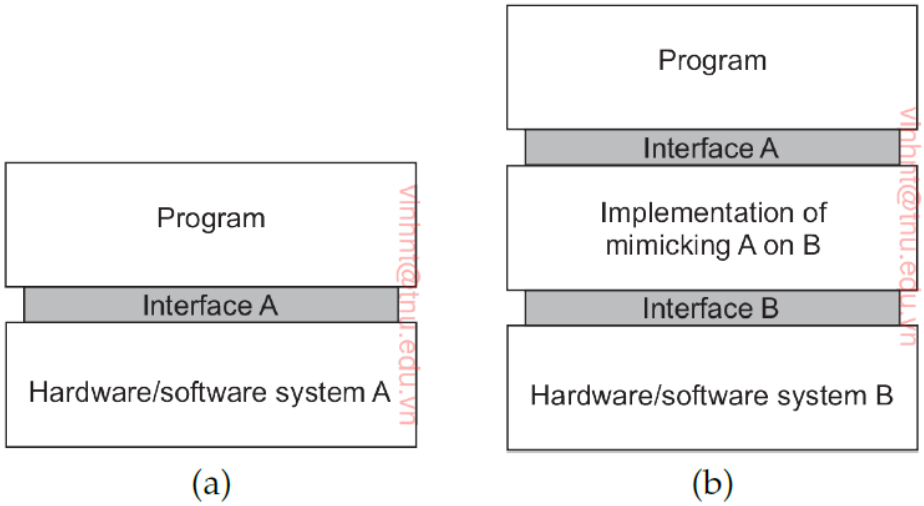


Figure 3.6: (a) General organization between a program, interface, and system. (b) General organization of virtualizing system A on top of B.

Process vs System Virtual Machines

- Two broad virtualization approaches:
  - **Process Virtual Machines**
  - **System Virtual Machines**
- Difference lies in:
  - Scope of virtualization
  - Level of hardware abstraction

Process Virtual Machines

- Virtualization for a single process
- Implemented using a runtime system
- Instructions may be:
  - Interpreted (e.g., Java Virtual Machine)
  - Emulated (e.g., Windows apps on Unix)
- Must also virtualize system calls

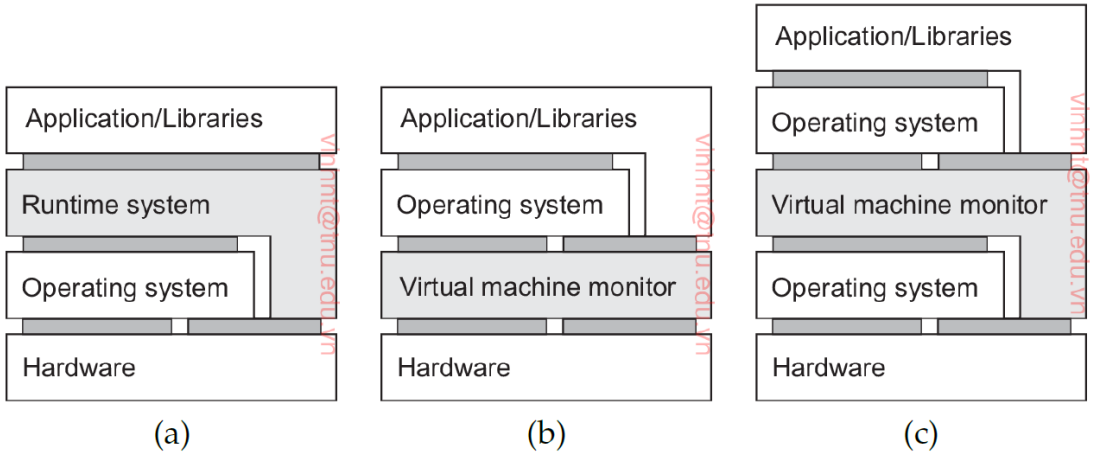


Figure 3.8: (a) A process virtual machine. (b) A native virtual machine monitor. (c) A hosted virtual machine monitor.

System Virtual Machines

- Virtualize the entire hardware platform
- Provide a complete instruction set
- Allow multiple guest OSs to run concurrently
- Implemented by a **Virtual Machine Monitor (VMM)**

Types of Virtual Machine Monitors

- **Native (Bare-Metal) VMM**
  - Runs directly on hardware
  - Manages CPU, memory, storage, network
- **Hosted VMM**
  - Runs on top of a host OS
  - Reuses host OS services and drivers

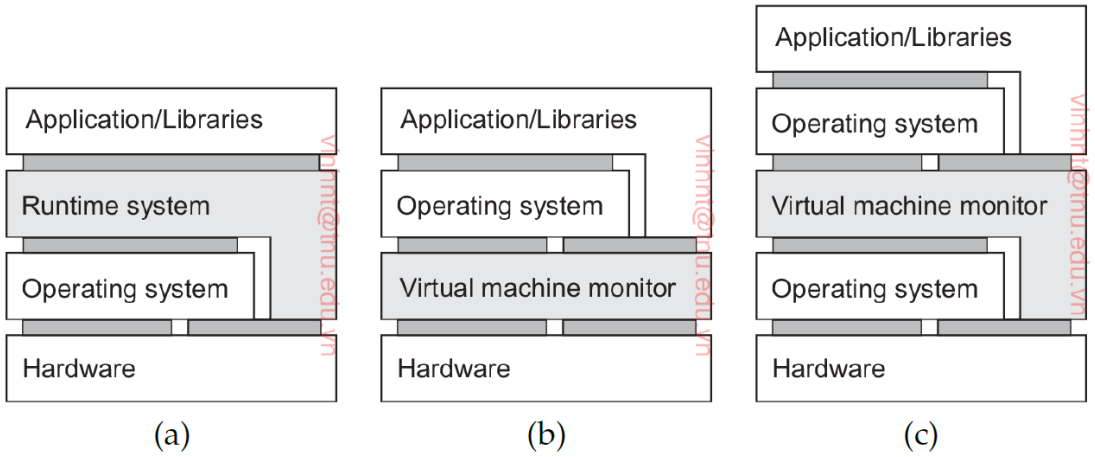


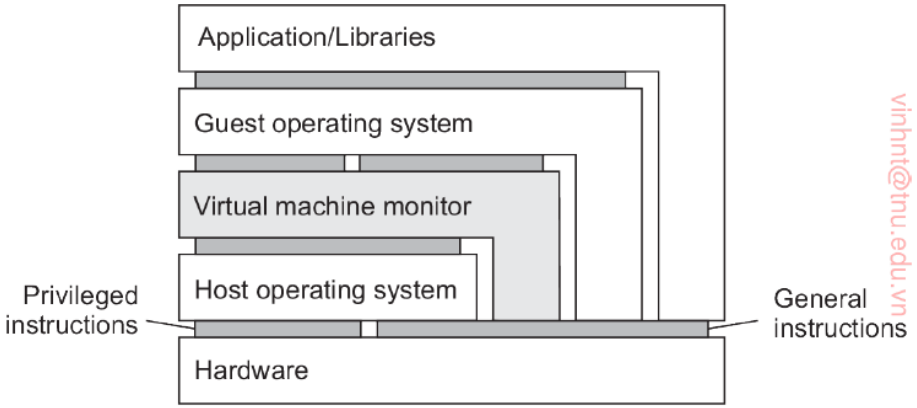
Figure 3.8: (a) A process virtual machine. (b) A native virtual machine monitor. (c) A hosted virtual machine monitor.

Why Virtual Machines Matter

- Provide strong isolation between environments
- Contain failures and security attacks
- Enable migration of entire systems
- Key benefits:
  - Portability
  - Flexibility
  - Fault isolation

Performance of Virtual Machines

- Modern VMs run close to native speed
- Most instructions execute directly on hardware
- Only privileged instructions trap to the VMM



vinhnt@tnu.edu.vn

Figure 3.9: Applications, guest operating system, virtual machine monitor, and host operating system on a single hardware platform.

The x86 Virtualization Challenge

- Classical virtualization requires:
  - Sensitive instructions to be privileged
- x86 violates this assumption:
  - Some sensitive instructions are not privileged
  - Execute without trapping in user mode

Solutions to x86 Virtualization

- **Full Virtualization**
  - Binary rewriting
  - Guest OS unchanged (e.g., VMware)
- **Paravirtualization**
  - Guest OS modified
  - Problematic instructions removed (e.g., Xen)

Virtualization and Cloud Computing

- Virtualization is the foundation of cloud computing
- Three service models:
  - Infrastructure-as-a-Service (IaaS)
  - Platform-as-a-Service (PaaS)
  - Software-as-a-Service (SaaS)
- Virtual machines are central to IaaS

Case Study: Amazon EC2

- Provides network-accessible virtual machines
- Uses Amazon Machine Images (AMIs)
- Each instance includes:
  - OS kernel
  - Preinstalled services and libraries
- Supports elastic resource configuration

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

Clients in Distributed Systems

- Clients are the **user-facing components** of a distributed system
- They provide access to **remote services**
- A client typically performs two roles:
  - User interaction (UI, input, output)
  - Communication with remote servers
- Main goal of a client:
  - Hide distribution aspects such as:
    - Network communication
    - Server location
    - Partial failures

Networked User Interfaces

- Clients must support interaction with remote services
- Two fundamental approaches exist:
  1. Application-specific networked interfaces
  2. General-purpose thin-client interfaces
- They differ mainly in:
  - Where application logic runs
  - Where data is stored

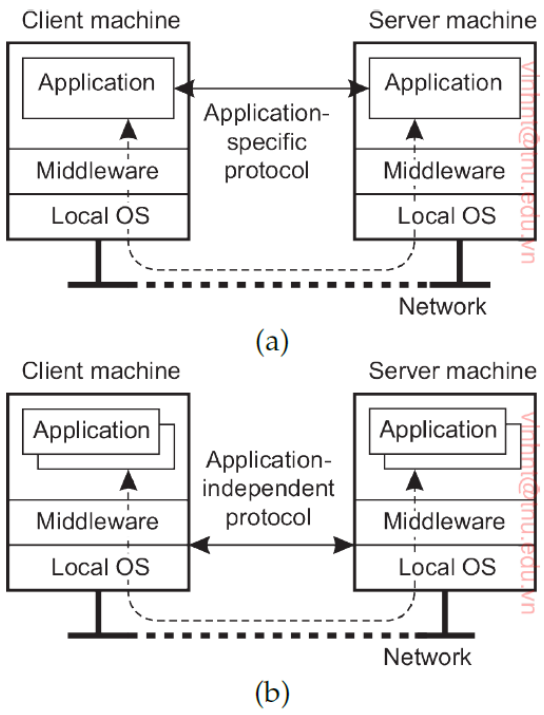


Figure 3.10: (a) A networked application with its own protocol. (b) A general solution to allow access to remote applications.

Application-Specific Networked Clients

- A dedicated client is developed for a specific service
- Communication uses an application-level protocol
- Client-side responsibilities include:
  - Managing the user interface
  - Executing part of the application logic
- Example:
  - A calendar app synchronizing with a remote calendar server

These clients are powerful but tightly coupled to the service.

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

Thin Clients

- The client machine behaves mainly as a terminal
- All application logic and data reside at the server
- Client responsibilities are limited to:
  - Displaying output
  - Forwarding user input
- Thin clients are:
  - Application-independent
  - Easier to manage and maintain

Thin Clients: Two Display Approaches

- Thin clients differ mainly in how displays are controlled
- Two common approaches:
  - **Command-based display protocols**
    - Example: X Window System
  - **Pixel-based display protocols**
    - Example: VNC
- In both cases:
  - Application logic runs at the server
  - Client handles input and display only

The X Window System

- One of the earliest networked user interface systems
- Designed to control bit-mapped displays
- Supports:
  - Display devices
  - Keyboards
  - Pointing devices
- Runs on desktops, workstations, and mobile devices
- Acts as the terminal-control component of an OS

Architecture of the X Window System

- Core component: **X kernel**
- Responsibilities:
  - Control display hardware
  - Capture input events
  - Perform hardware-dependent operations
- Applications access X via:
  - Xlib
  - Higher-level toolkits

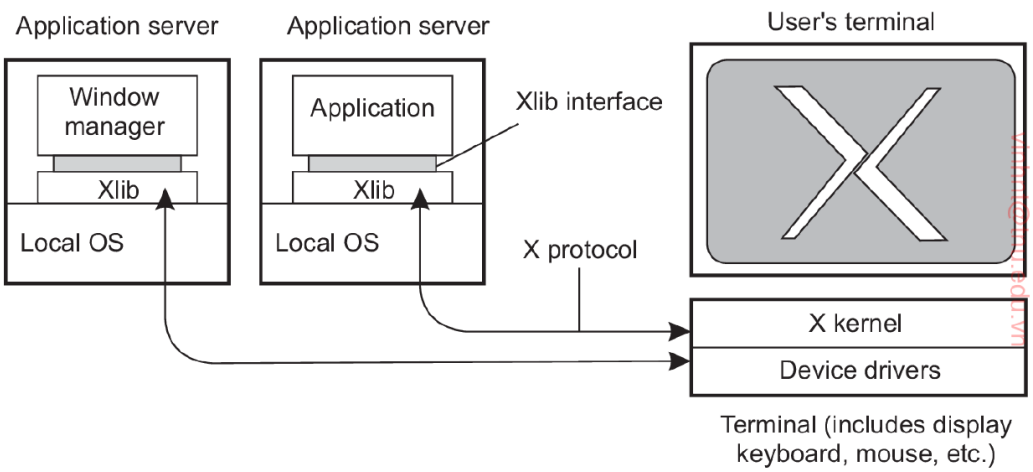


Figure 3.11: The basic organization of the X Window System.

X as a Client–Server System

- X applications and X kernel may run on different machines
- Communication uses the X protocol
- Role assignment:
  - X kernel provides display services (server)
  - Applications request these services (clients)

Thin Clients over Wide-Area Networks

- X tightly couples user interaction and application logic
- Leads to many synchronous request–response interactions
- Over wide-area networks, this causes:
  - High latency
  - Poor responsiveness
  - Bad user experience

Pixel-Based Remote Displays

- Alternative to command-based protocols like X
- Display is controlled at pixel level
- Bitmap updates are sent over the network
- Client updates its local frame buffer
- Example:
  - Virtual Network Computing (VNC)

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

Client-Side Middleware

- Client software includes middleware components
- Middleware hides distribution complexity
- Responsibilities:
  - Communication handling
  - Data marshalling and unmarshalling
  - Failure detection and recovery
- Goal:
  - Make remote interactions appear local

Replication Transparency

- Servers are often replicated
- Replication improves:
  - Fault tolerance
  - Performance
- Middleware hides replication from clients

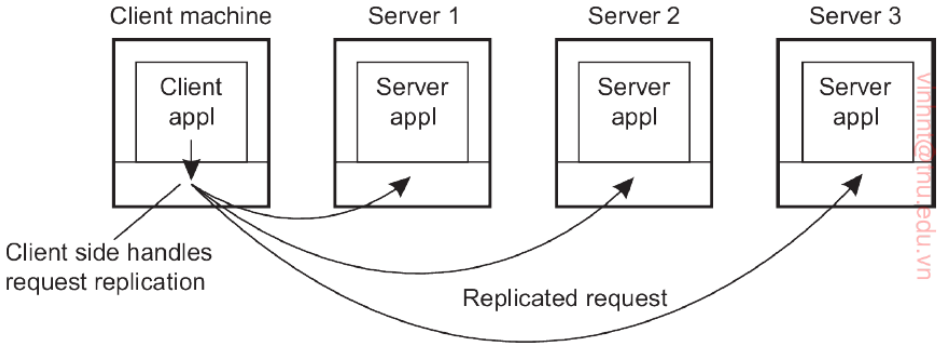


Figure 3.12: Transparent replication of a server using a client-side solution.

Types of Distribution Transparency

- **Access transparency**: remote calls look local
- **Location transparency**: server location is hidden
- **Migration transparency**: servers may move
- **Replication transparency**: multiple servers appear as one
- **Failure transparency**: failures are masked
- **Concurrency transparency**: safe concurrent access

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

Servers: Overview

- A server is a process that provides a service to multiple clients
- Servers wait for requests, process them, and respond
- This section discusses:
  - General server design issues
  - Object servers
  - Server clusters

Basic Server Organization

- Servers repeatedly perform the following steps:
  1. Wait for a client request
  2. Process the request
  3. Send a response
  4. Wait for the next request
- Key design concern:
  - How to handle requests efficiently

Iterative vs Concurrent Servers

Iterative Servers

- Handle one request at a time
- Next request is accepted only after completion
- Simple design
- Poor performance under heavy load

Concurrent Servers

- Delegate requests to threads or processes
- Main server immediately waits for new requests
- Better performance and scalability
- Examples:
  - Multithreaded servers
  - Fork-based servers in Unix

Server organization strongly affects performance and scalability.

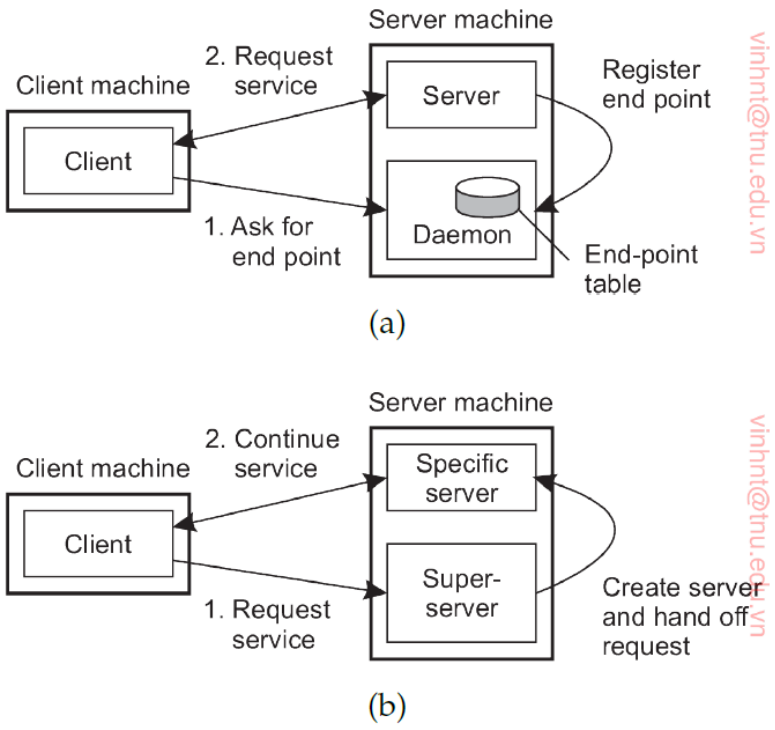
Contacting a Server: End Points

- Clients communicate with servers via **end points (ports)**
- Each server listens on a specific end point
- To contact a service, a client must know:
  - Network address of the server machine
  - End point associated with the service
- Types of end points:
  - **Well-known end points**:
    - FTP: TCP port 21
    - HTTP: TCP port 80
  - Assigned by the Internet Assigned Numbers Authority (IANA)
  - **Dynamically assigned end points**:
    - Assigned at runtime by the operating system

Client–Server Binding Using a Daemon

Daemon: a background process that manages and provides information about running services.

- The daemon maintains the current end points of services
- It listens on a well-known end point
- Client interaction steps:
  1. Client contacts the daemon
  2. Daemon returns the server's end point
  3. Client contacts the server directly



vinhnt@tnu.edu.vn

Figure 3.13: (a) Client-to-server binding using a daemon. (b) Client-to-server binding using a superserver.

Superservers

- Running many idle servers wastes resources
- A superserver listens on multiple service end points
- When a request arrives:
  - Superserver starts the required server
  - Server exits after handling the request
- Example:
  - inetd in Unix systems

Interrupting a Server: Out-of-Band Communication

- **Controlled interruption**:
  - A mechanism that allows a client to stop or control an ongoing server operation
  - The connection is not abruptly terminated
- **Out-of-band communication**:
  - Control messages that are handled with higher priority than normal data
- Why this is needed:
  - Clients may want to cancel long-running operations
  - Example: stopping a large file transfer
- **Example: FTP**
  - FTP uses a separate **control connection** and a **data connection**
  - Commands such as `ABOR` (abort) are sent on the control connection
  - The server interrupts the data transfer without closing the session
- Alternative mechanism:
  - TCP urgent (out-of-band) data sent on the same connection

Stateless vs Stateful Servers

Stateless Servers

- Do not store client-specific state
- Each request is handled independently
- Server can restart without recovery
- Simple fault tolerance
- Example:
  - Web servers (HTTP)

Soft State

- Temporary state maintained by the server
- State expires unless refreshed by the client
- Common in network protocols and subscriptions

Stateful Servers

- Maintain persistent client-related state
- State must be explicitly created and deleted
- Improves performance and efficiency
- Requires complex recovery after failures
- Example:
  - File servers with client caching

The stateless vs stateful choice strongly affects performance, fault tolerance, and recovery complexity.

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

Object Servers: Overview

- Object servers host distributed objects
- The server itself does not implement services
- Services are provided by hosted objects
- The server supports object invocation

Object Server Organization

- Objects consist of:
  - State (data)
  - Methods (code)
- The object server is responsible for:
  - Activating objects when needed
  - Handling invocation requests from clients

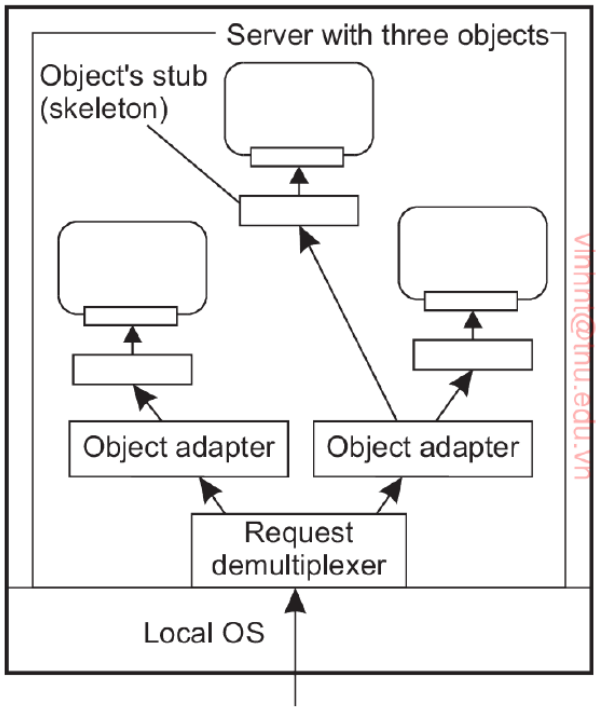


Figure 3.14: An object server supporting different activation policies.

Object Adapters

- **Object adapter:**
  - A server-side component that controls how objects are created and invoked
- Objects may require different activation and concurrency policies
- Responsibilities of an object adapter:
  - Activate objects when requests arrive
  - Dispatch invocation requests to the correct object
  - Apply activation and threading policies
- Key properties:
  - Generic and configurable (independent of object interfaces)
  - Multiple adapters can coexist in one server

Ice Runtime

An Ice object server is not a specialized OS entity; it is a standard process.

- **No Special Requirements:** Runs on standard hardware and operating systems.
- **The Transformation:** A normal process becomes an **object server** only by initializing the Ice runtime.

Logical Foundation

Object servers = Ordinary Processes + Library-based Runtime Support.

The Communicator: Resource Manager

The **Communicator** is the root of the Ice runtime, managing the "inner workings" of the server.

- **Resource Control:** Manages thread pools and memory allocation.
- **Configuration:** Defines behavior like max message size and retry limits.
- **Isolation:** While a server usually has one communicator, multiple can be used to isolate different applications within the same process.

Why Isolate?

Separate thread pools ensure that one overloaded application doesn't starve the resources of another in the same process.

Ice Runtime Architecture: Client and Server Structure

Component	Primary Responsibility	Side	Origin
Ice Core	Networking, threading, and byte-ordering.	Both	**Library**
Proxies	**Down-calls:** Translates local calls to RPC. Handles Marshaling.	Client	**Generated**
Skeletons	**Up-calls:** Transfers control from runtime to application code.	Server	**Generated**
Object Adapter	Maps requests to Servants; manages Endpoints (TCP/UDP).	Server	**Library/API**
Servants	Actual business logic (The code *you* write).	Server	**Programmed**

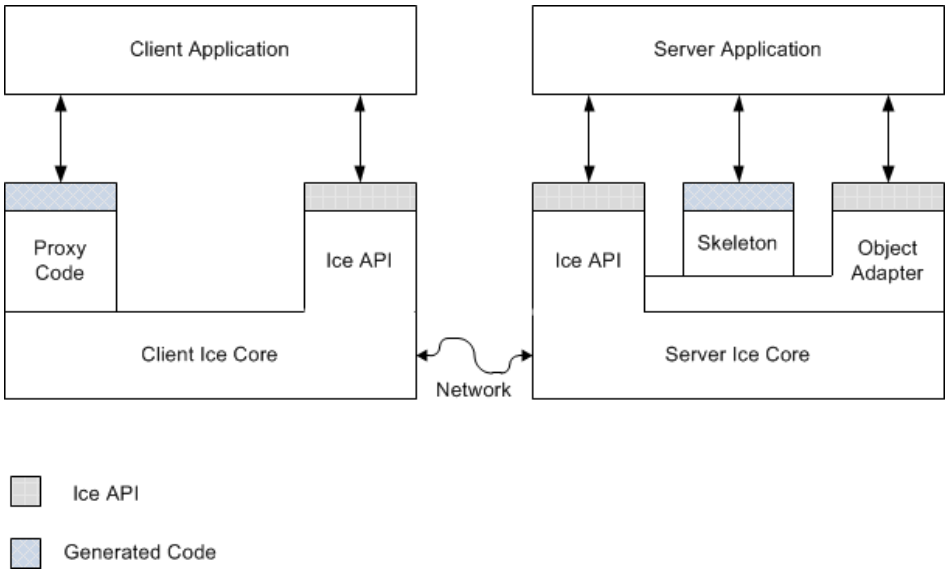


Figure 1: Request flow through the Ice stack.

Object Adapter: The Dispatcher

Created by the communicator, the **Object Adapter** bridges the network and your code.

- **Endpoints:** Binds the server to a specific protocol/port (e.g., TCP 10000).
- **Request Mapping:** Receives network packets and identifies which servant object should handle them.
- **Policy Enforcement:** Controls concurrency (e.g., single-threaded vs. multi-threaded adapters).

Implementation: From Interface to Execution

1. The Interface Definition (MyObject.ice)

```
interface MyObject {
 void doWork();  // This defines the method for both sides
};
```

2. Server (The Servant)

```
class MyObjectI : public MyObject {
 public:
 // Actual logic happens here
 void doWork(const Ice::Current&) override {
  std::cout << "Working..." << std::endl;
 }
};

int main(int argc, char* argv[]) {
 Ice::CommunicatorHolder ich(argc, argv);
 auto adapter = ich->createObjectAdapterWithEndpoints(
 "MyAdapter", "tcp -p 10000");
 adapter->add(std::make_shared<MyObjectI>(),
 Ice::stringToIdentity("uniqueID"));
 adapter->activate();
 ich->waitForShutdown();
}
```

3. Client (The Proxy)

```
int main(int argc, char* argv[]) {
 Ice::CommunicatorHolder ich(argc, argv);

 // Proxy points to Identity + Endpoint
 auto base = ich->stringToProxy(
 "uniqueID:tcp -p 10000");

 auto printer = MyObjectPrx::checkedCast(base);

 if(printer) {
  // Method is called on the Proxy
  printer->doWork();
 }
}
```

Deconstructing the Ice Implementation

The implementation follows a 3-step lifecycle: **Contract**, **Hosting**, and **Invocation**.

1. **The Slice Contract:** The `interface` acts as a language-independent bridge. Both the client and server agree on the `doWork()` signature before any code is written.
2. **Server-Side (The Implementation):**
   - **The Servant:** `MyObjectI` provides the actual logic.
   - **Registration:** `adapter->add` links the servant to a **Unique ID** (e.g., "uniqueID").
   - **Activation:** `adapter->activate()` opens the network port (`tcp 10000`) to start listening for requests.
3. **Client-Side (The Access):**
   - **The Proxy:** `stringToProxy` creates a local handle that "knows" where the server lives (ID + IP/Port).
   - **The Cast:** `checkedCast` verifies that the remote object actually supports the `MyObject` interface.
   - **The Call:** `printer->doWork()` looks like a local function call but is actually an RPC.

The "Identity" Connection

The key link between both sides is the string **"uniqueID"**. This is how the Object Adapter knows exactly which servant instance should handle the incoming network packet.

Scalability: The "Static" Problem

The Issue with `adapter->add()`

In our implementation, we manually registered `"uniqueID"`. Imagine a system with **100,000 users**:

- **Manual Labor:** You would need to call `adapter->add()` 100,000 times in a loop before calling `activate()`.
- **Memory Bloat:** The **Object Adapter** must keep all 100,000 servants in its "Active Servant Map" (RAM) simultaneously, even if only 5 users are online.
- **Slow Boot:** The server is stuck initializing those objects instead of responding to requests.

The Solution: Servant Locators (Dynamic)

- Instead of `adapter->add()`, you register a **Servant Locator**.
- The **Object Adapter** stays empty until a request arrives.
- The **Communicator** only allocates memory for a servant when it's actually needed.

Solution: Database-Driven Scaling

- **Storage:** Object states live in a database (e.g., user profiles).
- **Mapping:** The Object ID acts as the Database Primary Key.
- **Process:**
  1. Request arrives $\rightarrow$ 2. Locator queries DB $\rightarrow$ 3. Object instantiated $\rightarrow$ 4. Request processed.

This allows a single adapter to virtually manage millions of objects with minimal RAM usage.

From Ice to Enterprise Java Beans

Ice demonstrates how object servers can be built using a runtime library.

- The programmer explicitly manages:
  - Object adapters
  - Activation policies
  - Resource usage
- Enterprise Java Beans (EJB) take a different path

Key Shift

Instead of a library, EJB places objects inside a **managed server environment**.

Same object-server idea, but with stronger server-side control.

What Is an Enterprise Java Bean?

An Enterprise Java Bean (EJB) is a Java object that runs inside an application server.

- Clients invoke methods on the object remotely
- The server automatically provides:
  - Transaction management
  - Security
  - Persistence and resource access

Separation of Concerns

- Application code: business logic
- Server infrastructure: system-level services

EJB Container: Managed Object Adapter

- Each EJB executes inside a **container**
- The container:
  - Creates and destroys EJB objects
  - Manages transactions and concurrency
  - Provides naming, messaging, and persistence services
- Services are injected automatically by the server

Developers write business logic; the container handles everything else.

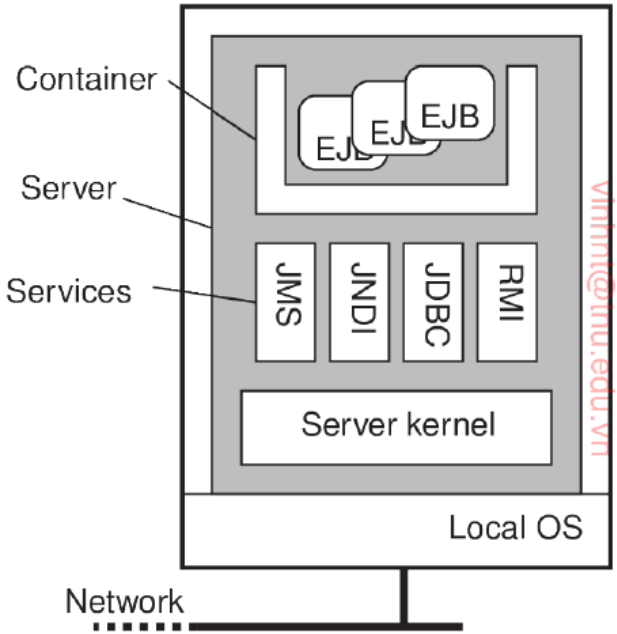


Figure 3.16: General architecture of an EJB server.

EJB Types: Managing State

EJBs are categorized based on how they manage state.

- **Stateless Session Beans**
  - No client-specific state
  - Each invocation is independent
- **Stateful Session Beans**
  - Maintain state across multiple client calls
  - Exist for the duration of a client session
- **Entity Beans**
  - Represent persistent data stored in databases
  - Survive beyond individual client sessions
- **Message-Driven Beans**
  - Respond to asynchronous messages
  - Do not maintain conversational state

EJB Implementation: From Interface to Execution

1. The Interface Definition (MyObject.java)

```
@Remote
public interface MyObject {
 void doWork();  // Standard Java interface with @Remote annotation
};
```

2. Server (The Bean)

```
@Stateless
public class MyObjectBean implements MyObject {
 @Override
 public void doWork() {
  System.out.println("Working...");
 }
}

/* NOTE: NO main() method!
The Container automatically:
- Picks a port
- Manages threads
- Listens for requests
- Activates the object */
```

3. Client (The Lookup)

```
public class Client {
 public static void main(String[] args) {
  Context ctx = new InitialContext();

  // Lookup replaces stringToProxy
  // Uses JNDI name instead of IP/Port
  MyObject bean = (MyObject) ctx.lookup(
  "ejb:/app/MyObjectBean!MyObject");

  // Method call is intercepted by Container
  bean.doWork();
 }
}
```

Ice vs Enterprise Java Beans: Conceptual View

Concept	**Ice**	**EJB**
Object server	Explicit object server created by application	Implicit object server managed by application server
Adapter	Object Adapter	Container
Lifecycle	Application-controlled	Server-controlled
Services	Explicit APIs used by programmer	Automatically provided by container
Philosophy	Library-based distributed system	Server-managed component model

Both systems support object servers, but differ in where control is placed.

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

The Apache Web Server

Apache is a widely used Web server and a good example of separating **mechanisms** from **policies**.

- Extremely popular and widely deployed
- Highly configurable and extensible
- Largely independent of the underlying operating system

Key Design Goal

Provide a flexible server core that can be extended without modifying the core itself.

Platform Independence: Apache Portable Runtime

Apache achieves platform independence using the **Apache Portable Runtime (APR)**.

- APR is a runtime library that provides:
  - File handling
  - Networking
  - Threads and locking
- APR hides operating-system differences

Practical Rule

Extensions remain portable as long as they use APR and avoid platform-specific system calls.

Apache as a Request-Processing Server

Apache can be viewed as a general server that processes requests and produces responses.

- Primarily designed for Web documents
- Assumes:
  - HTTP as the application protocol
  - TCP as the transport protocol
- UDP-based requests are not supported without modifying the core

Important Observation

Apache fixes some mechanisms (e.g., TCP), but leaves request handling policies flexible.

Apache Architecture: Hooks and Modules

Apache processes requests in a sequence of well-defined phases.

- Each phase consists of one or more **hooks**
- A hook is a placeholder for a group of functions
- Hooks are executed in a predefined order

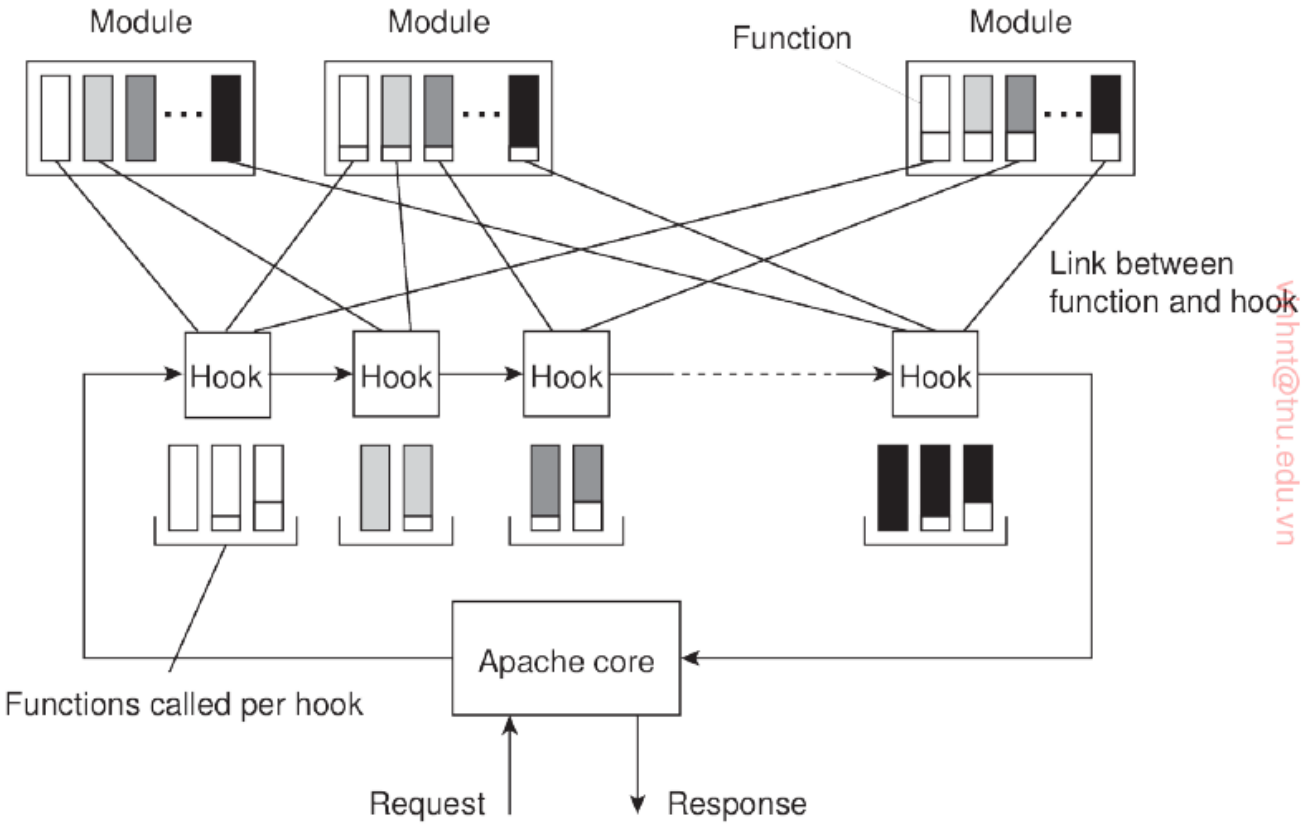


Figure 3.17: The general organization of the Apache Web server.

vsthnt@tnu.edu.vn

Modules: Implementing Policies in Apache

Apache functionality is implemented using **modules**.

- Each module provides functions for specific hooks
- Functions must match predefined prototypes
- Multiple modules can register functions for the same hook

Execution Order

- Functions in a hook may execute in arbitrary order
- Ordering constraints can be specified if needed

Result: a highly extensible and configurable Web server.

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

Server Clusters: Overview

- A server cluster is a collection of machines
- Each machine runs one or more servers
- Goals:
  - Scalability
  - Availability

Three-Tier Server Cluster Architecture

- Server clusters are often organized into:
  - Front-end switch
  - Application servers
  - Data servers

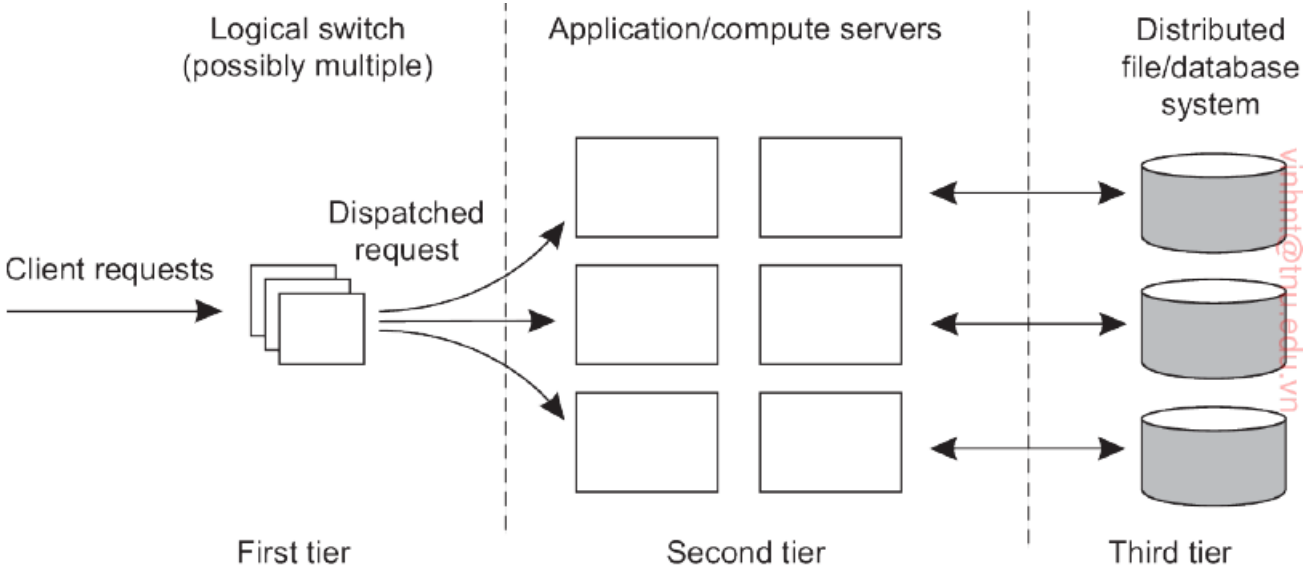


Figure 3.18: The general organization of a three-tiered server cluster.

vinhnt@tnu.edu.vn

Request Dispatching

- The switch provides a single access point
- Clients are unaware of internal structure
- Requests are forwarded to selected servers

TCP Handoff

- TCP handoff is used in server clusters with a front-end switch
- The client initiates a TCP connection to the switch
- The switch selects a suitable server
- The TCP connection is **handed off** to that server
- After handoff:
  - Requests go to the server
  - Responses go **directly** to the client
- The switch is removed from the data path
- This significantly reduces load on the switch

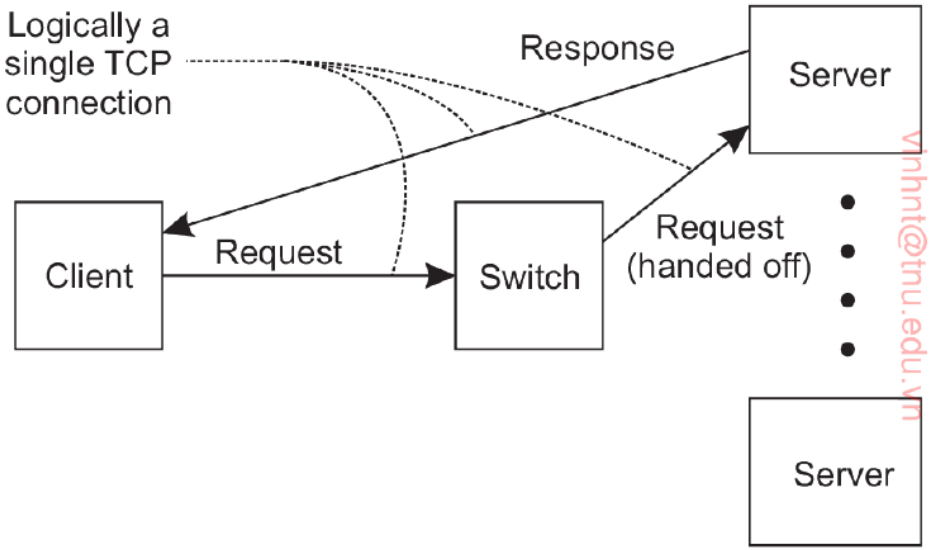


Figure 3.19: The principle of TCP handoff.

vinhnt@tnu.edu.vn

Wide-Area Server Clusters: DNS-Based Redirection

- **Wide-area server clusters**:
  - Servers are distributed across geographically distant locations
  - Commonly deployed using cloud computing infrastructure
- **Primary goal**:
  - Improve locality and reduce client-perceived latency
- **DNS-based request redirection**:
  - DNS returns different server IP addresses for the same service name
  - Clients are redirected to a nearby or suitable server
  - Redirection is transparent to the client
- **Limitation**:
  - Client location may be estimated inaccurately using DNS
- Real-world examples include CDNs used by YouTube, Netflix, and AWS

Case Study: PlanetLab

Let's look at a very different kind of server cluster.

- PlanetLab is a **collaborative distributed system**
- Many organizations donate computers
- All donated machines work together

Important Observation

PlanetLab forms a **1-tier server cluster**:

- Access
- Processing
- Storage

all happen on the same nodes.

Because nodes are globally distributed, centralized control is impractical.

PlanetLab Node: Basic Organization

Each PlanetLab node follows the same internal design.

- Two key components:
  - **Virtual Machine Monitor (VMM)**
  - **Linux Vservers**
- The VMM is a modified Linux OS
- Vservers run on top of the VMM

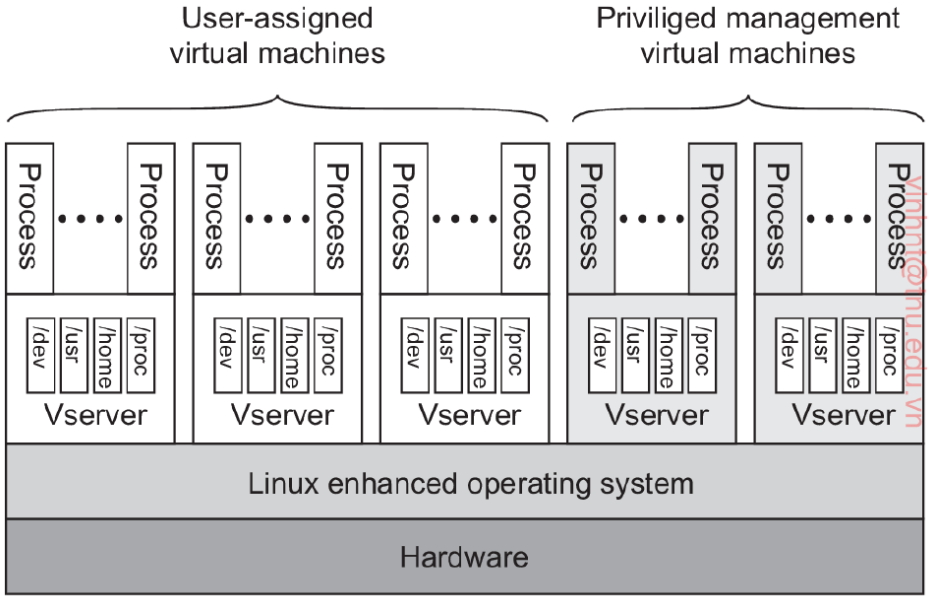


Figure 3.22: The basic organization of a PlanetLab node.

Think of the VMM as the referee and Vservers as isolated players.

Vservers: Isolated Execution Environments

A Vserver is a container that runs a group of processes.

- Many Vservers can run on the same physical node
- Processes in different Vservers:
  - Do not interfere
  - Are isolated from each other
- Even identical user IDs remain separated

Why Is This Important?

It allows users from different organizations to safely share the same machine.

Slices: Virtual Server Clusters

PlanetLab enables experiments using **slices**.

- A slice is a set of Vservers
- Each Vserver runs on a different node
- Together, they behave like a server cluster

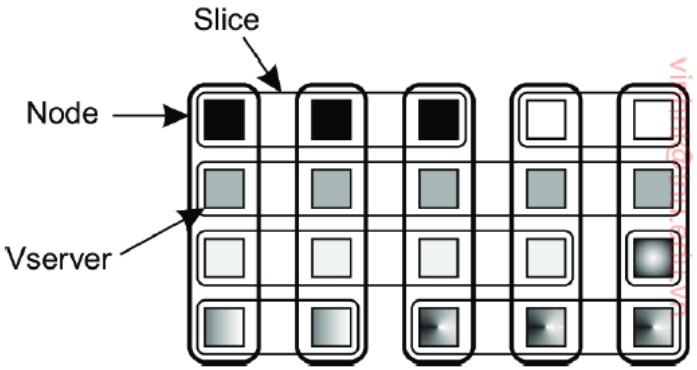


Figure 3.23: The principle of a PlanetLab slice, showing sets of associated Vservers across different nodes.

A slice is essentially a virtual server cluster built from containers.

Resource Management in PlanetLab

Resource control is handled locally on each node.

- Every node runs a **node manager**
- The node manager:
  - Creates Vservers
  - Allocates CPU, memory, disk, and bandwidth
- Node managers are not accessible remotely

How Resources Are Tracked

- Resources are assigned to slices
- Allocation is described using an **rspec**
- Each rspec has a unique **resource capability (rcap)**

Why Container-Based Virtualization?

PlanetLab uses containers instead of full virtual machines.

- All Vservers share the same OS kernel
- Only user-space environments are separated
- Isolation is achieved using:
  - Separate namespaces
  - Protected directory trees

Key Advantage

- Dynamic resource allocation
- Memory and CPU overbooking
- Many Vservers can run on low-memory machines

If a Vserver misbehaves, it is simply reset.

Key Takeaways

- Server organization affects scalability and performance
- Stateless servers simplify recovery
- Object servers separate policy from mechanism
- Server clusters hide distribution behind a single access point

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

Conceptual Overview

- **Traditional Model:** Communication is restricted to the exchange of **data**.
- **Migration Model:** The **program code** is transferred between nodes.
- **Execution:** The relocated code executes on the remote target machine.

The Paradigm Shift

Transitioning from "bringing data to the program" to "moving the program to the data."

Motivation for Code Migration

Optimization Criterion: Bandwidth vs. Latency

- **Volume-driven:** Move code when the dataset is too large to transfer efficiently.
- **Interaction-driven:** Move code to reduce high-frequency network round-trips.

Primary Objective

To minimize overall network communication overhead and improve system performance.

Historical Context: Process Migration

Legacy distributed systems attempted to move active, **running processes**.

- **Initial Goal:** Dynamic load balancing across CPU nodes.
- **Implementation Challenges:**
  - Excessive computational overhead.
  - High complexity in maintaining process state.
  - Difficulties in ensuring reliability across heterogeneous hosts.

Note: Pure process migration is largely superseded by modern virtualization.

Modern Implementation: VM Migration

Contemporary systems prioritize the migration of **Virtual Machines (VMs)** over individual processes.

- **Advantages:**
  - Stronger isolation and cleaner abstractions.
  - Simplifies state capture compared to process-level migration.
- **Industrial Use Cases:**
  - Energy-efficient resource management in data centers.
  - Dynamic workload consolidation and maintenance.

Scenario A: Data-Centric Migration

Problem: Accessing massive server-side databases.

- Transferring raw datasets to the client is bandwidth-intensive.
- **Solution:**
  - Dispatch the computation (query logic) to the server.
  - Perform local processing at the data source.
  - Return only the synthesized result to the client.

Standard practice in modern database and "Big Data" architectures.

Scenario B: Client-Side Offloading

Problem: Latency-sensitive interactive applications.

- Constant back-and-forth messaging degrades user experience.
- **Solution:**
  - Move application logic (e.g., form validation) to the client.
  - Execute interactions locally to mask network latency.
  - Synchronize with the server only for final state updates.

Outcome: Improved responsiveness and reduced server-side processing load.

Taxonomy of Code Mobility Paradigms

1. **Client–Server:** Code remains stationary at the server.
2. **Remote Evaluation:** Client dispatches code to the server.
3. **Code-on-Demand:** Client fetches code from the server.
4. **Mobile Agents:** Code and its execution state migrate autonomously.

Distinguished by the location of code and the site of execution.

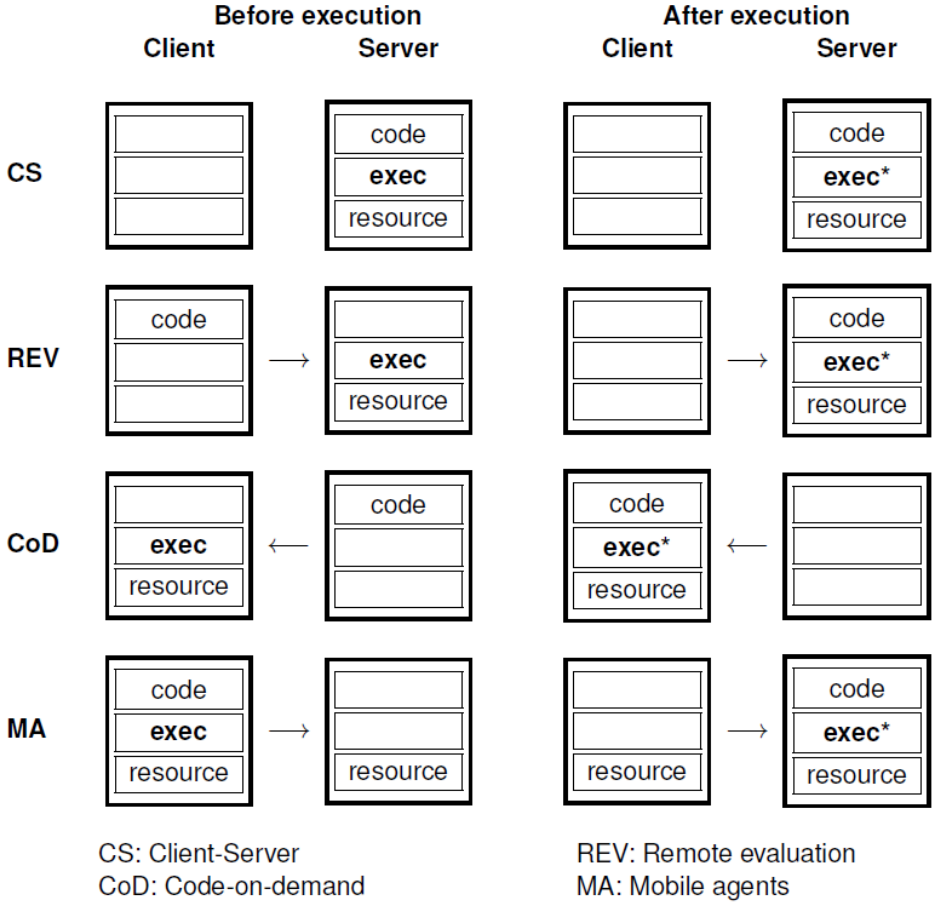


Figure 3.25: Four different paradigms for code mobility.

Mobility Models: Weak vs. Strong

Weak Mobility

- Transfers the code segment only.
- Execution always restarts from a predefined entry point.
- Low complexity; widely used in practice (e.g., Java Applets, JavaScript).

Strong Mobility

- Transfers both code and full execution context (stack, registers).
- Execution resumes precisely from the point of interruption.
- Significant implementation complexity and overhead.

Operational Semantics: Migration vs. Cloning

Migration

- The process is relocated to a target node.
- The original instance is terminated upon successful transfer.

Cloning

- A replica of the process is created on the target node.
- Both instances continue execution concurrently.
- Analogous to a networked implementation of `fork()`.

Challenges: System Heterogeneity

Physical hardware and software environments vary across nodes.

- **Barriers:** Differing CPU architectures and operating system kernels.
- **Technical Solutions:**
  - Hardware abstraction via Virtual Machines.
  - Platform-independent languages (e.g., interpreted languages or bytecode).

VM Migration Strategies

Three primary mechanisms for state transfer:

1. **Stop-and-Copy:** Simple execution pause; results in high downtime.
2. **Demand Paging:** Immediate migration; fetches memory pages on-demand (high latency).
3. **Pre-copy:** Iterative background transfer of memory pages before the final switch.

Conclusion: Pre-copy is the standard for "Live Migration" in production environments.

Key Takeaways

- Code migration optimizes computation by improving proximity to resources.
- Communication reduction is typically more critical than CPU load balancing.
- VM-based migration is the dominant modern implementation.
- Strategy selection depends on the trade-off between downtime and network load.

Core Principle

Migrate code when the cost of data movement exceeds the cost of code movement.

Table of Contents

1. Introduction
2. Threads
3. Virtualization
4. Client
   - Thin Clients
     - X Window System
     - Pixel-Based Remote Displays
   - Client-Side Middleware
5. Servers
   - Object Servers
     - Ice Runtime System
     - Enterprise Java Beans
   - Example: The Apache Web Server
   - Server Clusters
     - Case Study: PlanetLab
6. Code Migration
7. Summary

1. Threads: Performance vs. Protection

- **The Core Trade-off:** Processes offer **isolation** (safe but slow IPC); Threads offer **performance** (shared memory but zero protection).
- **Context Switch Costs:**
  - *Direct:* Saving registers/stack. *Indirect:* Cache pollution and TLB flushes (The "hidden" cost).
- **Mapping Models:** Modern systems lean toward **Many-to-Many** or **One-to-One** to balance user-space speed with kernel-level parallelism.

Distributed Necessity

In DS, threads are primarily used to **mask communication latency**. While one thread blocks on a network I/O, others continue execution.

2. Virtualization: Bridging the Software-Hardware Gap

- **Process VMs:** Abstract a single execution environment (JVM, CLR).
- **System VMs:** Abstract the entire hardware (VMM/Hypervisor).
- **The x86 Hurdle:** Overcome by **Binary Translation** (Full Virtualization) or **Guest OS Modification** (Paravirtualization).

Cloud Impact

Virtualization is the operational foundation of **IaaS** (e.g., EC2), enabling rapid scaling, fault isolation, and hardware-independent migration.

3. Clients: Managing the User Experience

Thin Clients (VNC/X11)

- Logic runs on server.
- Minimal client hardware.
- Sensitive to network latency.

Fat Clients

- Logic runs locally.
- Complex synchronization.
- Better offline capabilities.

- **Middleware Role:** Client-side stubs/proxies handle the "Heavy Lifting" of **Transparency** (Location, Replication, and Failure masking).

4. Servers: State and Scalability

- **The State Dilemma:**
  - **Stateless:** Highly scalable, easy recovery (HTTP).
  - **Stateful:** Better performance for complex sessions but difficult to recover.
- **Server Clusters:**
  - **TCP Handoff:** Removes the front-end switch from the data path to prevent bottlenecks.

Object Servers (Ice vs. EJB)

Transition from **Library-based** (explicit management) to **Container-managed** (implicit management) object lifecycles.

5. Code Migration: Strategic Mobility

- **The "Why":** It is often cheaper to move **KB of code** to the data than to move **GB of data** to the code.
- **Mobility Taxonomy:**
  - **Weak:** Code only (starts from `main()`).
  - **Strong:** Code + Stack (resumes execution).
- **Live Migration: Pre-copy** is the industry standard—iteratively moving memory pages while the VM is still running.

Final Insight

Modern distributed systems rely on **Virtual Machines** as the primary unit of migration to overcome hardware/OS heterogeneity.